

Empirical Software Engineering: A gentle introduction

Roberto Verdecchia
University of Florence



UNIVERSITÀ
DEGLI STUDI
FIRENZE

DINFO

DIPARTIMENTO DI
INGEGNERIA DELL'INFORMAZIONE



**Software
Technologies
Laboratory**



This is an interactive lecture!
I will ask you questions, and
you should do the same!

A pinch about me



A pinch about me

∞ Meta

KPMG



SIG



Schiphol
Group

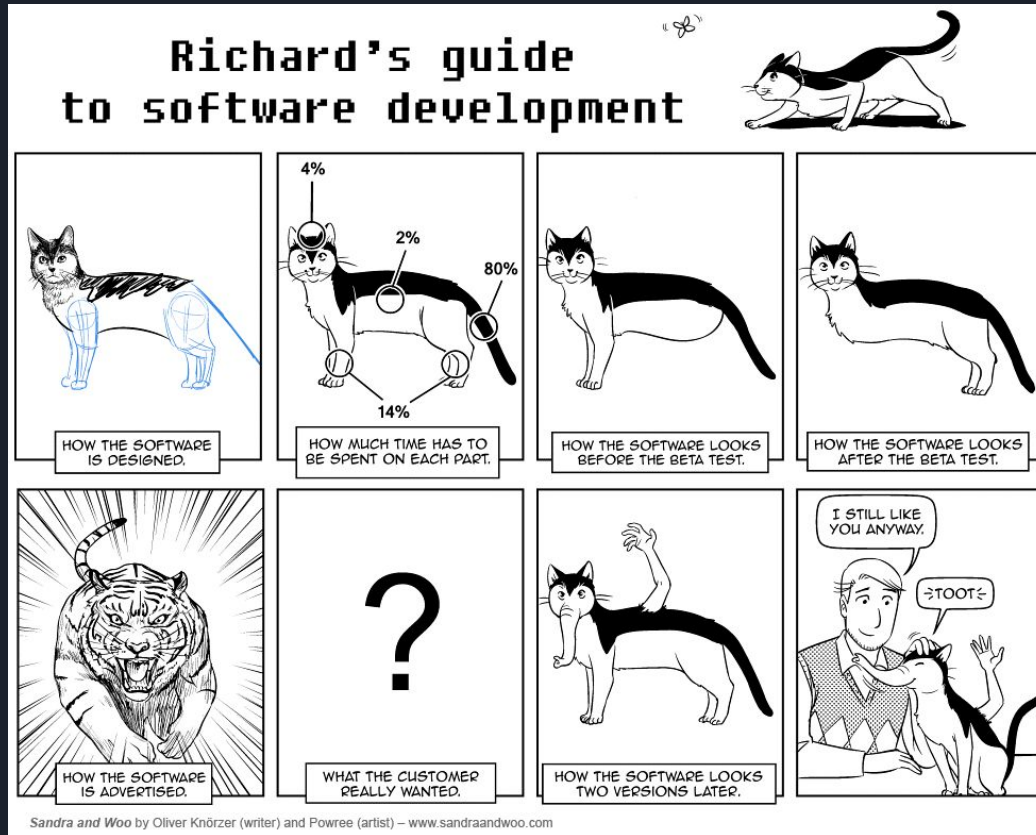


42

SDIA



Empirical SE: Predicting the future



Empirical Software Engineering: The What

Scientific use of quantitative and qualitative data to understand and improve software products and software development processes

[Victor Basili]

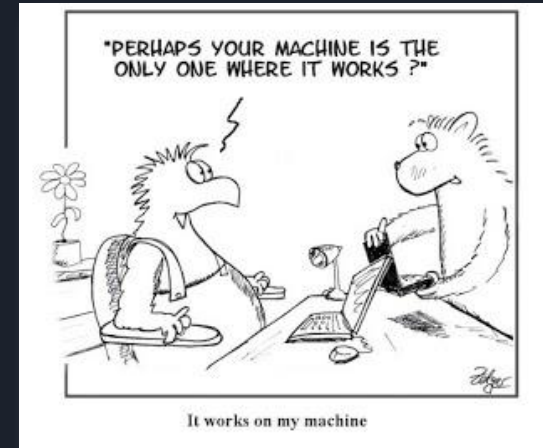
Data is central to address any problem

Issues related to validity addressed continuously

Empirical Software Engineering: The Why

Anecdotal evidence or “**common-sense**” often **not good enough**:

- **Anecdotes often insufficient** to support decisions
- **Evidence-based** software development
- Trust that solutions will **work in general**
- Search for the **proven best solution**



Empirical Software Engineering: The Why

- *Scientifically assess an idea*
- *Generalize*

Empirical Software Engineering: The Why



“What will be the rating of this movie?”



“What if half of my servers crashed?”



“How can I identify who is a bot and who is human?”

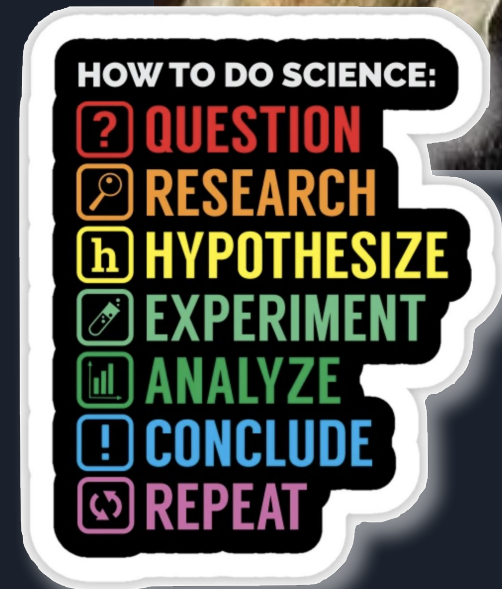
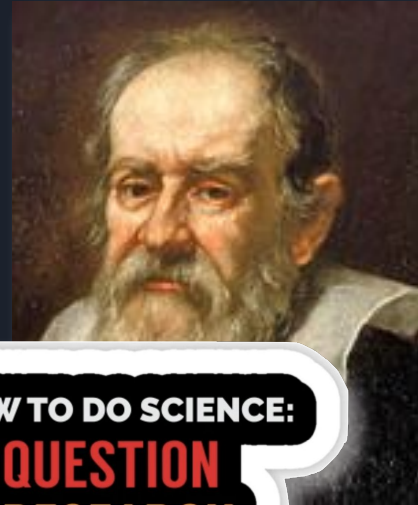
Can you think of other
examples where an
empirical experiment is
needed?

Empirical Software Engineering: The how

Usually starts from an **intuition** 💡

Application of the **scientific method**

1. Ask yourself a **question**
2. **Background** research
3. Formulate an **hypothesis**
4. Set up an **experiment**
5. **Observe** phenomenon
6. Perform **analysis** on your results
7. Draw **conclusions**



Dimensions of empirical studies

“In the lab” versus **“in the wild”** studies

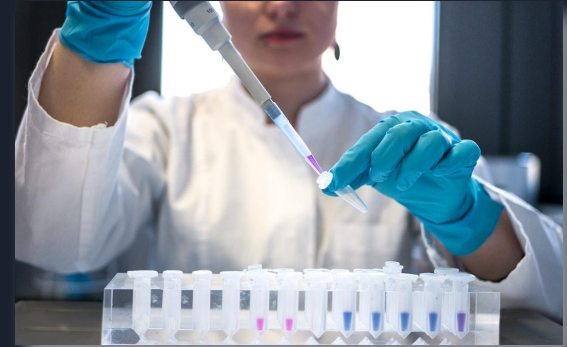
Quantitative versus **qualitative** studies

Inductive and **deductive** studies

“In the lab” versus “in the wild” studies

Common “in the lab” methods

- Controlled experiments
- Literature reviews
- Simulations



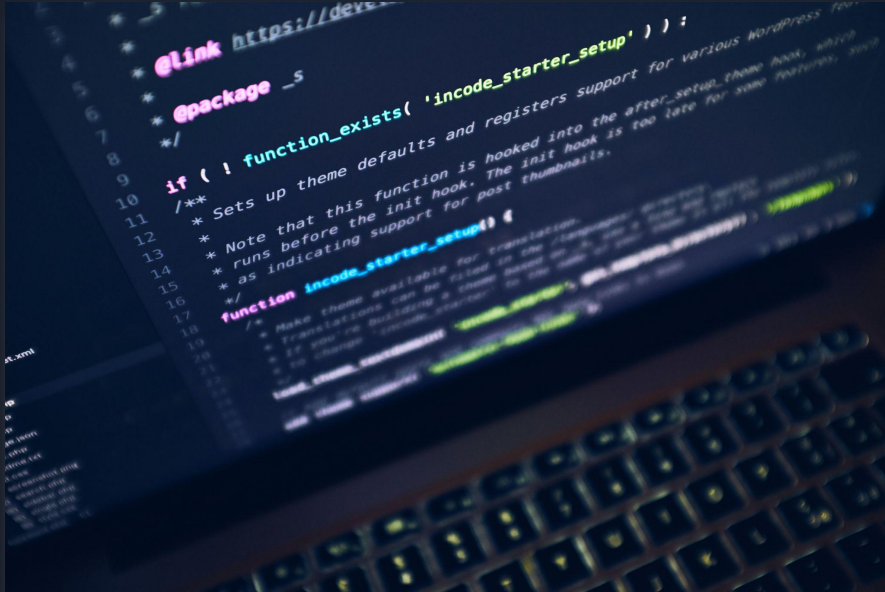
Common “in the wild” methods

- Quasi-experiments
- Case studies
- Survey research
- Ethnographies
- Action research



Can you give an example of
“in the lab” and “in the wild”
software engineering
study?

Quantitative vs Qualitative Studies



Quantitative vs Qualitative Studies

Quantitative research

Quantifying a relationship or to **compare two or more groups** with the aim to identify a **cause-effect relationship**

- fixed implied factors
- focus on quantitative data → promotes **comparison** and **statistical analyses**

Qualitative research

Studying objects in their **natural setting** and letting the **findings emerge from the observations**

- **mostly inductive** process (usually discovers the “why”)
- the **subject is the person**

Is this a qualitative or quantitative study?

Hashing It Out: A Survey of Programmers' Cannabis Usage, Perception, and Motivation

Madeline Endres
endremad@umich.edu
University of Michigan
Ann Arbor, Michigan, USA

Kevin Boehnke
kboehnke@umich.edu
University of Michigan
Ann Arbor, Michigan, USA

Westley Weimer
weimerw@umich.edu
University of Michigan
Ann Arbor, Michigan, USA

ABSTRACT

Cannabis is one of the most common mind-altering substances. It is used both medicinally and recreationally and is enmeshed in a complex and changing legal landscape. Anecdotal evidence suggests that some software developers may use cannabis to aid some programming tasks. At the same time, anti-drug policies and tests remain common in many software engineering environments, sometimes leading to hiring shortages for certain jobs. Despite these connections, little is actually known about the prevalence of, and motivation for, cannabis use while programming. In this paper, we report the results of the first large-scale survey of cannabis use by programmers. We report findings about 803 developers' (including 450 full-time programmers') cannabis **usage prevalence, perceptions, and motivations**. For example, we find that some programmers *do* regularly use cannabis while programming: 35% of our sample has tried programming while using cannabis, and 18% currently do so at least once a month. Furthermore, this cannabis usage is primarily motivated by a perceived enhancement to certain software development skills (such as brainstorming or getting into a programming zone) rather than medicinal reasons (such as pain relief). Finally, we find that cannabis use while programming occurs at similar rates for programming employees, managers, and students despite differences in cannabis perceptions and visibility. Our results have implications for programming job drug policies and motivate future research into cannabis use while programming.

CCS CONCEPTS

• **Software and its engineering** → *Software development techniques*; • **Social and professional topics** → *Technology policy*.

1 INTRODUCTION

Cannabis sativa (hereafter, cannabis) is the world's most commonly used illicit substance, used by more than 192 million people in 2018 [54]. The global cannabis market in 2020 was estimated at 20.5 billion USD, and is estimated to grow to 90.4 billion by 2026 [65]. Globally, cannabis legality is changing rapidly, with many countries (e.g., the United Kingdom, Colombia, and Malawi) legalizing medical cannabis, and a subset (e.g., Uruguay, Mexico, and Canada) also legalizing cannabis for recreational adult use.¹ In the US, for example, medical cannabis is allowed in 36 states, and 17 states have legalized cannabis² for adult use despite its federal classification as a Schedule I drug, which criminalizes cannabis and defines it as having no accepted therapeutic value and a high abuse potential [47]. This classification has hampered research on its therapeutic effects [47], and prohibition is contrary to popular opinion: 91% of Americans believe cannabis should be legal for medical or recreational purposes [56]. Similarly, 81% of Americans believe cannabis has at least one benefit [35]. These benefits are mostly medical, but 16% and 11% cited improved creativity and focus, respectively.

Despite legal concerns, anecdotes of cannabis use intersecting with software engineering abound. Questions inquiring about cannabis's effects on programming are common on online forums such as Reddit,³ Quora,⁴ Hacker News,⁵ and Dev,⁶ often inspiring numerous conflicting answers. Similarly, popular tech-related media sites cover the topic, with one iTechPost article claiming that "folks in the tech mecca that is Silicon Valley are clearly getting their fair share of medical (or otherwise) marijuana", positing that cannabis may help with chronic pain associated with long hours of programming [36]. This claim aligns with epidemiological trends, as chronic pain is the most common reason for medical cannabis licen-

ABSTRACT

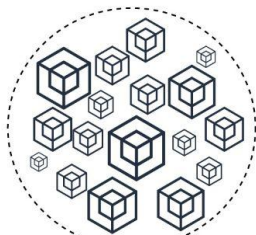
Cannabis is one of the most common mind-altering substances. It is used both medicinally and recreationally and is enmeshed in a complex and changing legal landscape. Anecdotal evidence suggests that some software developers may use cannabis to aid some programming tasks. At the same time, anti-drug policies and tests remain common in many software engineering environments, sometimes leading to hiring shortages for certain jobs. Despite these connections, little is actually known about the prevalence of, and motivation for, cannabis use while programming. In this paper, we report the results of the first **large-scale survey of cannabis use by programmers**. We report findings about **803 developers'** (including 450 full-time programmers') cannabis **usage prevalence, perceptions, and motivations**. For example, we find that some programmers *do* regularly use cannabis while programming: **35% of our sample has tried programming while using cannabis**, and 18% currently do so at least once a month. Furthermore, this cannabis usage is primarily **motivated by a perceived enhancement to certain software development skills** (such as brainstorming or getting into a programming zone) rather than medicinal reasons (such as pain relief). Finally, we find that cannabis use while programming **occurs at similar rates for programming employees, managers, and students** despite differences in cannabis perceptions and visibility. Our results have implications for programming job drug policies and motivate future research into cannabis use while programming.

Endres, M., Boehnke, K. and Weimer, W., "Hashing it out: a survey of programmers' cannabis usage, perception, and motivation". In Proceedings of the 44th International Conference on Software Engineering (ICSE). 2022

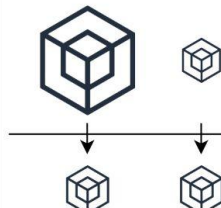
An example of quantitative research

Empirical evaluation of an architectural technical debt index in the context of the Apache and ONAP ecosystems

Roberto Verdecchia¹, James Maloney², Robert Winkler³ and Frank Ockers⁴



1. Gather measurements of architectural issues across a software portfolio



2. Normalize measurements to them across projects of different sizes

Results are then shared with the portfolio contributors, who are invited to participate in an online survey designed to evaluate the representativeness and actionability of the approach.

Results. The survey results confirm the representativeness of the ATDx, in terms of both the ATDx analysis results and the used architectural technical debt dimensions. Results also showed the actionability of the approach, although to a lower extent when compared to the ATDx representativeness, with usage scenarios including refactoring, code review, communication, and ATD evolution analysis.

Conclusions. With ATDx, we strive for the establishment of a sound, comprehensive, and intuitive architectural view of the ATD identifiable via source code analysis. The collected results are promising, and display both the representativeness and actionability of the approach. As future work, we plan to consolidate the approach via further empirical experimentation, by considering other development contexts (e.g., proprietary portfolios and other source code analysis tools), and enhancing the ATDx report capabilities.

Submitted 27 June 2021
Accepted 6 December 2021
Published 7 February 2022

Corresponding author
Roberto Verdecchia,
r.verdecchia@vu.nl

Academic editor
Robert Winkler

Additional Information and
Declarations can be found on
page 41

DOI 10.7717/peerj-cs.833

© Copyright
2022 Verdecchia et al.

Distributed under

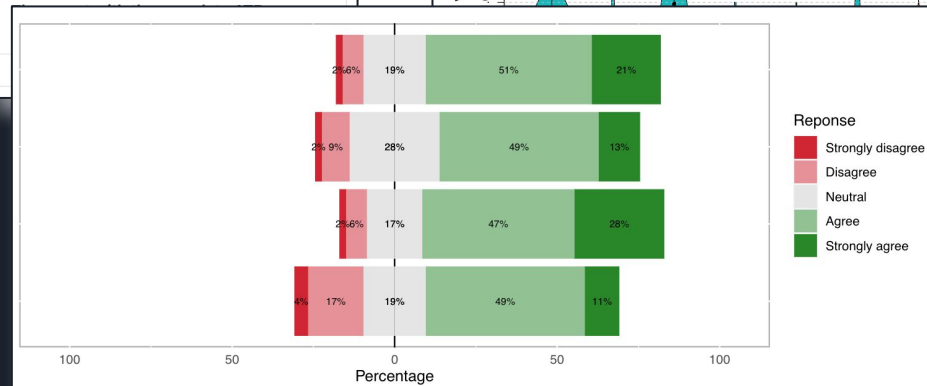
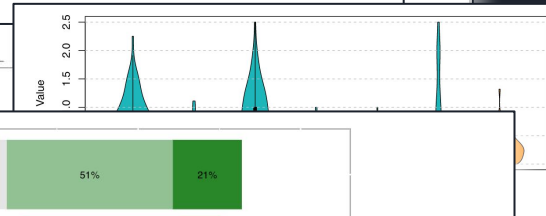
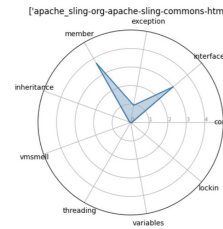
Subjects Computer Architecture, Software Engineering

Keywords Technical debt, Software architecture, Index, Software metrics, Software portfolio analysis, Empirical evaluation

ATDx-tool commented 1 minute ago

ATDx analysis results

The ATDx value is: 0.2



An example of qualitative research

The Journal of Systems & Software 176 (2021) 110925



The Journal

journal home

Building and evaluating a theory of software-intensive systems*

Roberto Verdecchia^a, Philippe Kruchten^{b, *}

^a Vrije Universiteit Amsterdam, The Netherlands

^b University of British Columbia, Vancouver, Canada

^c Chalmers University of Technology, Gothenburg, Sweden

ARTICLE INFO

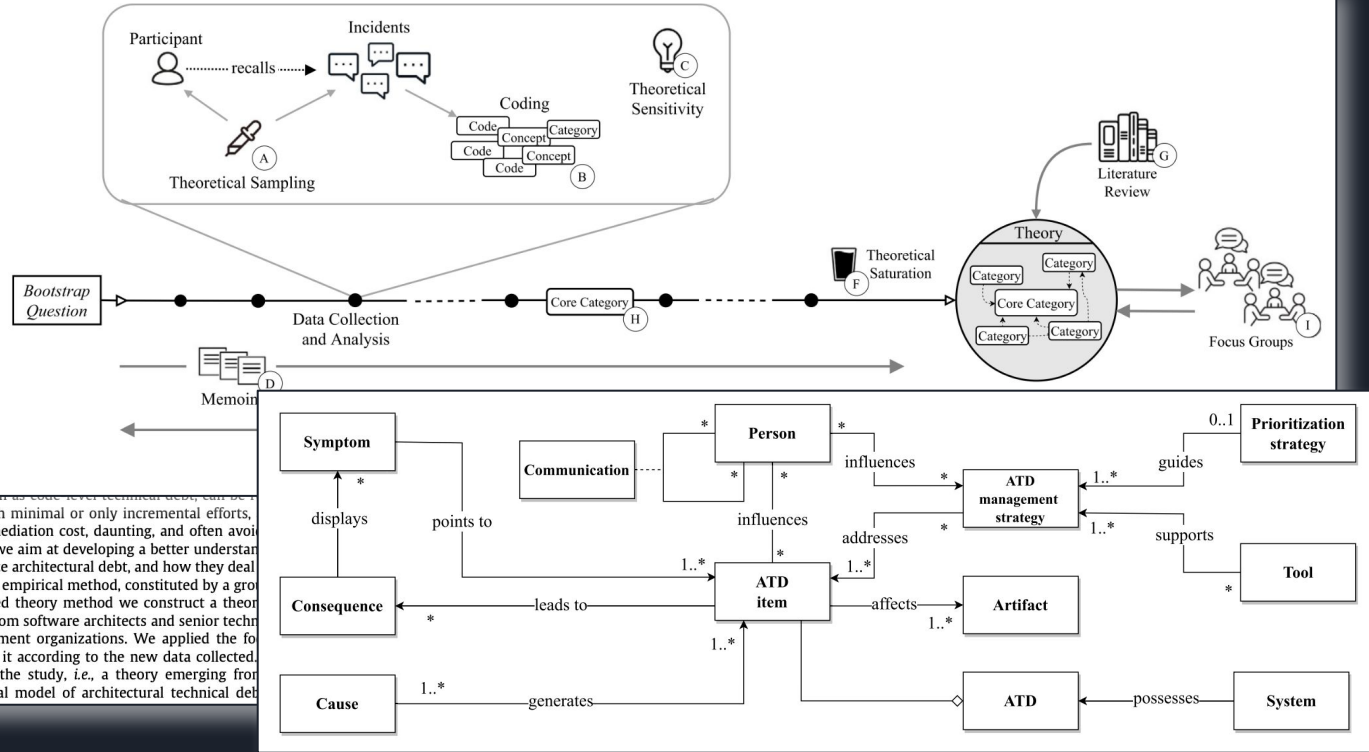
Article history:
Received 22 December 2020
Accepted 8 February 2021
Available online 27 February 2021

Keywords:

Software engineering
Software architecture
Technical debt
Software evolution
Grounded theory
Focus group

ABSTRACT

Architectural technical debt (ATD) is a design decision, being suitable of types of debt, such as code level technical debt, can be refactored with minimal or only incremental efforts, wide-ranging remediation cost, daunting, and often avoided. In this study, we aim at developing a better understanding of architectural technical debt, and how they deal with it. We apply a mixed empirical method, constituted by a grounded theory method. With the grounded theory method we construct a theory of architectural technical debt from software architects and senior technical software development organizations. We applied the theory and refine it according to the new data collected. The result of the study, i.e., a theory emerging from passing conceptual model of architectural technical debt.



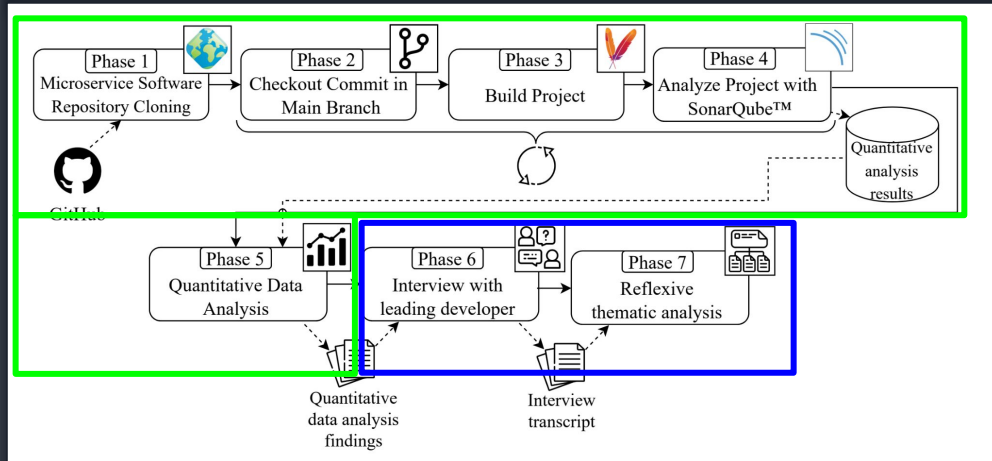
Qualitative and quantitative methods can be complementary!

Technical Debt in Microservices: A Mixed-Method Case Study

Roberto Verdecchia^{*[0000-0001-9206-6637]}, Kevin Maggi^{*},
Leonardo Scommegna^[0000-0002-7293-0210], and Enrico Vicario^[0000-0002-4983-4386]

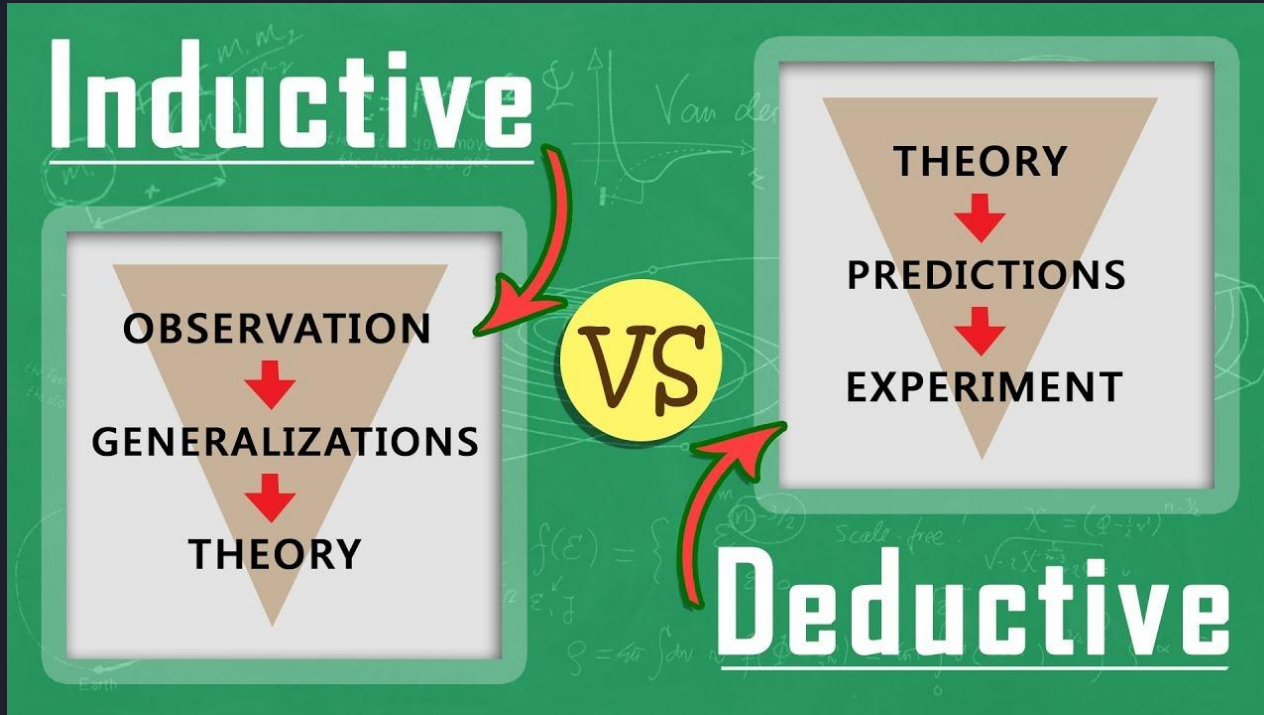
Department of Information Engineering, University of Florence, Italy
roberto.verdecchia@unifi.it,
kevin.maggi@edu.unifi.it, leonardo.scommegna@unifi.it, enrico.vicario@unifi.it

Abstract. *Background:* Despite the rising interest of both academia and industry in microservice-based architectures and technical debt, the landscape remains uncharted when it comes to exploring the technical debt evolution in software systems built on this architecture. *Aims:* This study aims to unravel how technical debt evolves in software-intensive systems that utilize microservice architecture, focusing on (i) the patterns of its evolution, and (ii) the correlation between technical debt and the number of microservices. *Method:* We employ a mixed-method case study on an application with 13 microservices, 977 commits, and 38k lines of code. Our approach combines repository mining, automated code analysis, and manual inspection. The findings are discussed with the lead developer in a semi-structured interview, followed by a reflexive thematic analysis. *Results:* Despite periods of no TD growth, TD generally increases over time. TD variations can occur irrespective of microservice count or commit activity. TD and microservice numbers are often correlated. Adding or removing a microservice impacts TD similarly, regardless of existing microservice count. *Conclusions:* Developers must be cautious about the potential technical debt they might introduce, irrespective of the development activity conducted or the number of microservices involved. Maintaining steady technical debt during prolonged periods of time is possible, but growth, particularly during innovative phases, may be unavoidable. While monitoring technical debt is the key to start managing it, technical debt code analysis tools must be used wisely, as their output always necessitates also a qualitative system understanding to gain the complete picture.



What is the difference
between inductive and
deductive thinking?

Inductive and deductive reasoning



Examples of popular empirical studies types: **Observational studies**

- **Collect and analyze software-related artefacts** to get a better understanding of a certain **phenomenon**
- Frequently qualitative studies (e.g., focus groups and case studies) or quantitative based on repository mining
- **Software repository mining**
 - a. **Select** software repositories (e.g., Android apps)
 - b. **Checkout** repositories (usually version per version)
 - c. Calculate **software metrics**
 - d. Draw **conclusions**

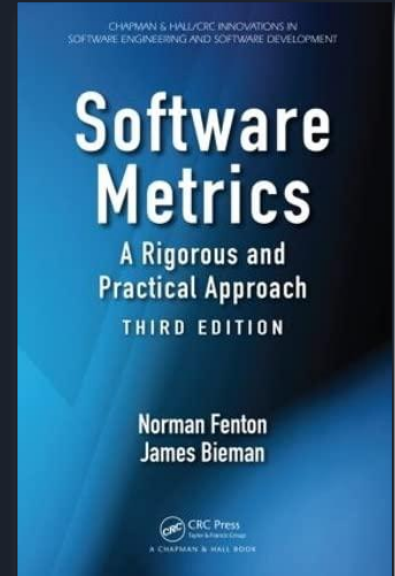


Brief excursus: **Software Metrics**

**What is a software
metric?**

Brief excursus: **Software Metrics**

- **Measure** of a **software artefact property**, usually based on source code analysis
- Can be either
 - **Static**, i.e., based on code parsing (e.g., SLOC)
 - **Dynamic**, i.e., calculated at runtime (e.g., execution time)
- Examples:
 - Code Churn
 - Cyclomatic complexity
 - Number of bugs per SLOC
 - Energy consumption
 - ...?



Examples of popular empirical studies types: **Observational studies**

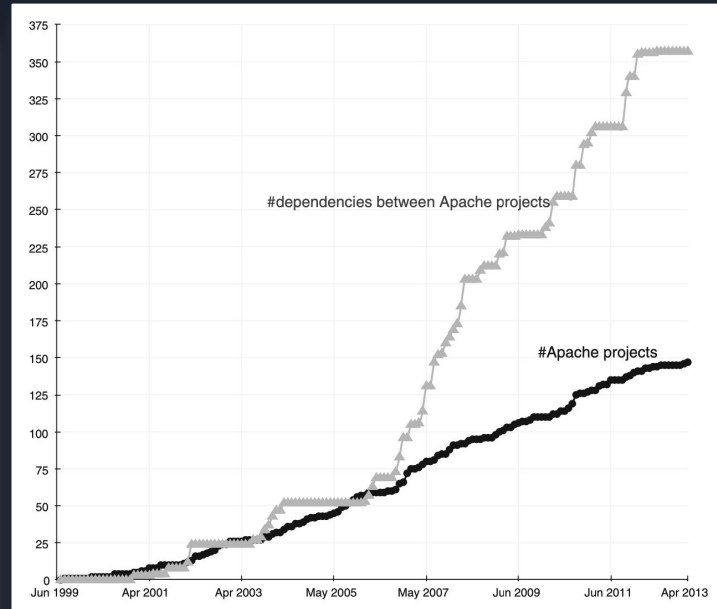
Empir Software Eng (2015) 20:1275–1317
DOI 10.1007/s10664-014-9325-9

How the Apache community upgrades dependencies: an evolutionary study

Gabriele Bavota · Gerardo Canfora ·
Massimiliano Di Penta · Rocco Oliveto ·
Sebastiano Panichella

Published online: 21 September 2014
© Springer Science+Business Media New York 2014

Abstract Software ecosystems consist of multiple software projects, often interrelated by means of dependency relations. When one project undergoes changes, other projects may decide to upgrade their dependency. For example, a project could use a new version of a component from another project because the latter has been enhanced or subject to some bug-fixing activities. In this paper we study the evolution of dependencies between projects in the Java subset of the Apache ecosystem, consisting of 147 projects, for a period of 14 years, resulting in 1,964 releases. Specifically, we investigate (i) how dependencies between projects evolve over time when the ecosystem grows, (ii) what are the product and process factors that can likely trigger dependency upgrades, (iii) how developers discuss the needs and risks of such upgrades, and (iv) what is the likely impact of upgrades on client projects. The study results—qualitatively confirmed by observations made by analyzing the



Examples of popular empirical studies types: **Observational studies**

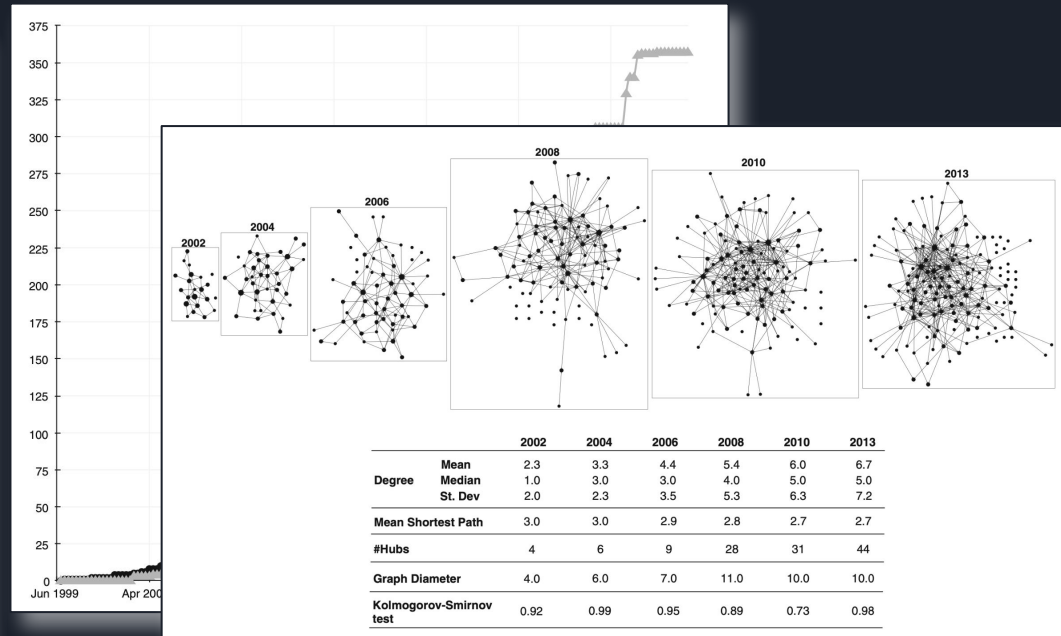
Empir Software Eng (2015) 20:1275–1317
DOI 10.1007/s10664-014-9325-9

How the Apache community upgrades dependencies: an evolutionary study

Gabriele Bavota · Gerardo Canfora ·
Massimiliano Di Penta · Rocco Oliveto ·
Sebastiano Panichella

Published online: 21 September 2014
© Springer Science+Business Media New York 2014

Abstract Software ecosystems consist of multiple software projects, often interrelated by means of dependency relations. When one project undergoes changes, other projects may decide to upgrade their dependency. For example, a project could use a new version of a component from another project because the latter has been enhanced or subject to some bug-fixing activities. In this paper we study the evolution of dependencies between projects in the Java subset of the Apache ecosystem, consisting of 147 projects, for a period of 14 years, resulting in 1,964 releases. Specifically, we investigate (i) how dependencies between projects evolve over time when the ecosystem grows, (ii) what are the product and process factors that can likely trigger dependency upgrades, (iii) how developers discuss the needs and risks of such upgrades, and (iv) what is the likely impact of upgrades on client projects. The study results—qualitatively confirmed by observations made by analyzing the



Examples of popular empirical studies types: **Observational studies**

Empir Software Eng (2015) 20:1275–1317
DOI 10.1007/s10664-014-9325-9

How the Apache community upgrades dependencies: an evolutionary study

Gabriele Bavota · Gerardo Canfora ·
Massimiliano Di Penta · Rocco Oliveto ·
Sebastiano Panichella

Published online: 21 September 2014
© Springer Science+Business Media New York 2014

Abstract Software ecosystems consist of multiple software projects, often interrelated by means of dependency relations. When one project undergoes changes, other projects may decide to upgrade their dependency. For example, a project could use a new version of a component from another project because the latter has been enhanced or subject to some bug-fixing activities. In this paper we study the evolution of dependencies between projects in the Java subset of the Apache ecosystem, consisting of 147 projects, for a period of 14 years, resulting in 1,964 releases. Specifically, we investigate (i) how dependencies between projects evolve over time when the ecosystem grows, (ii) what are the product and process factors that can likely trigger dependency upgrades, (iii) how developers discuss the needs and risks of such upgrades, and (iv) what is the likely impact of upgrades on client projects. The study results—qualitatively confirmed by observations made by analyzing the

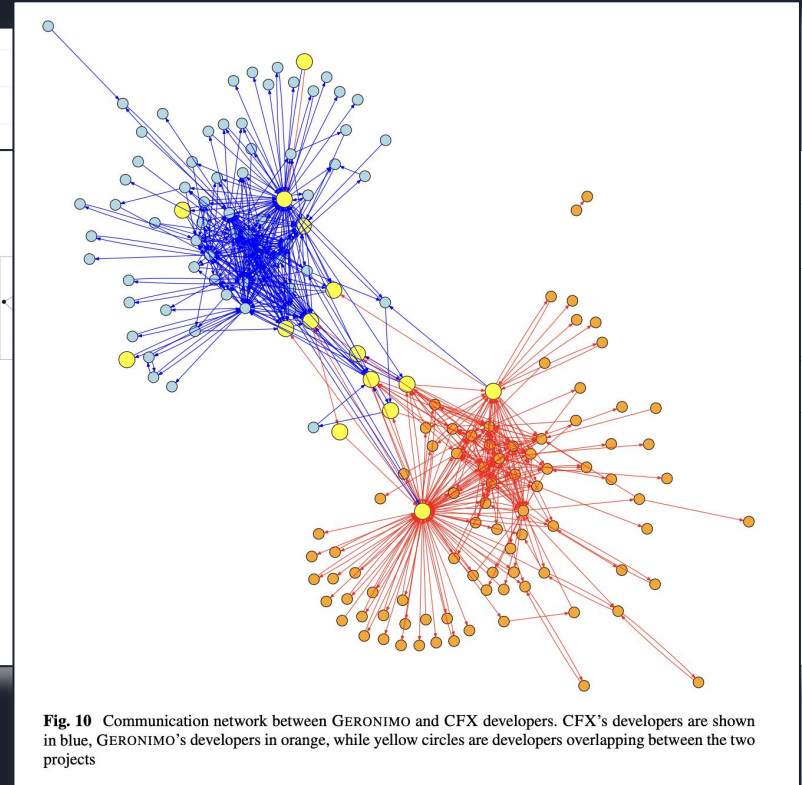
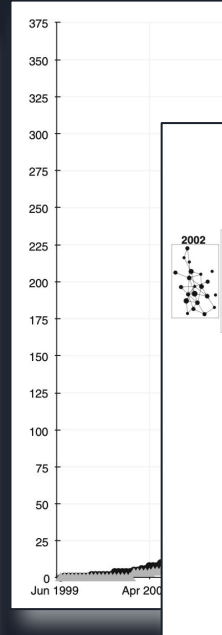


Fig. 10 Communication network between GERONIMO and CFX developers. CFX's developers are shown in blue, GERONIMO's developers in orange, while yellow circles are developers overlapping between the two projects

**How could they have evaluated
the potential impact of library
upgrades on client projects?**

Examples of popular empirical studies types: **Algorithm Evaluation**

- Empirical **evaluation** of a **new algorithm / solution** to prove its **efficiency, effectiveness**, and (usually) **improvement over the state of the art**
- Primarily based on **controlled experiments**
 - a. **Design** an algorithm / solution to address an open SE problem
 - b. **Select** existing “**competitor**” **algorithms / solutions** (if any exist)
 - c. **Compare** the **algorithms / solutions** via a **controlled experiment**



Examples of popular empirical studies types: **Algorithm Evaluation**

Scalable Approaches for Test Suite Reduction

Emilio Cruciani*, Breno Miranda†§, Roberto Verdecchia*‡, and Antonia Bertolino§

*Gran Sasso Science Institute | L'Aquila, Italy

†Federal University of Pernambuco | Recife, Brazil

‡Vrije Universiteit Amsterdam | Amsterdam, The Netherlands

§ISTI – Consiglio Nazionale delle Ricerche | Pisa, Italy

*emilio.cruciani@gssi.it | †bafm@cin.ufpe.br | ‡roberto.verdecchia@gssi.it | §antonia.bertolino@isti.cnr.it

Abstract—Test suite reduction approaches aim at decreasing software regression testing costs by selecting a representative subset from large-size test suites. **Most existing techniques are too expensive for handling modern massive systems** and moreover depend on artifacts, such as code coverage metrics or specification models, that are not commonly available at large scale. **We present a family of novel very efficient approaches** for similarity-based test suite reduction that apply algorithms borrowed from the big data domain together with smart heuristics for finding an evenly spread subset of test cases. The approaches are very general since they only use as input the test cases themselves (test source code or command line input). **We evaluate four approaches** in a version that selects a fixed budget B of test cases, and also in an adequate version that does the reduction guaranteeing some fixed coverage. The results show that **the approaches yield a fault detection loss comparable to state-of-the-art techniques**, while providing **huge gains in terms of efficiency**. When applied to a suite of more than 500K real world test cases, the most efficient of the four approaches could select B test cases (for varying B values) in less than 10 seconds.

Index Terms—Clustering, Random projection, Similarity-based testing, Software testing, Test suite reduction.

T' the changes made to S are considered (*modification-aware* regression testing) or not [34].

The proposed techniques have been evaluated and compared against each other using metrics relative to their fault detection effectiveness (e.g., the Average Percentage of Fault Detection of the reordered test suite, or the loss in faults detected by the reduced test suite T'); for test reduction and selection, also metrics relative to cost savings, e.g., the size or the execution time of T' are compared against those of the full suite T .

Another important factor that should be taken into account is the *cost of the technique itself*, both in terms of the computational effort and of the resources it requires. In other words, when evaluating whether investing on an automated approach aimed at reducing the cost of testing is worth, a complete cost-benefit analysis should also include the overheads implied by the approach [18].

However, not many of the proposed techniques have considered such implied costs. In 2004, Orso and coauthors already noticed that in regression testing efficiency and precision need

Algorithm 1 FAST++

Input: Test Suite T ; Budget B

Output: Reduced Test Suite R

```
1:  $P \leftarrow \text{RandomProjection}(T)$  ▷ Preparation phase
2:  $s \leftarrow \text{FirstSelection}(P)$ 
3:  $R \leftarrow \text{List}(s)$ 
4:  $D \leftarrow \text{Distance}()$  ▷ Squared distance to closest point in R
5:  $D(s) \leftarrow 0$ 
6: while ( $\text{Size}(R) < B$ ) do
7:   for all  $t \in P$  do
8:     if  $d(P(t), P(s))^2 < D(t)$  then
9:        $D(t) \leftarrow d(P(t), P(s))^2$  ▷ Squared Euclidean distance
10:   $s \leftarrow \text{ProportionalSample}(P, D)$ 
11:   $R \leftarrow \text{Append}(R, s)$ 
12:   $D(s) \leftarrow 0$ 
13: return  $R$ 
```

Examples of popular empirical studies types: **Algorithm Evaluation**

Scalable Approaches for Test Suite Reduction

Emilio Cruciani*, Breno Miranda†§, Roberto Verdecchia*‡, and Antonia Bertolino§

*Gran Sasso Science Institute | L'Aquila, Italy

†Federal University of Pernambuco | Recife, Brazil

‡Vrije Universiteit Amsterdam | Amsterdam, The Netherlands

§ISTI – Consiglio Nazionale delle Ricerche | Pisa, Italy

*emilio.cruciani@gssi.it | †bafm@cin.ufpe.br | ‡roberto.verdecchia@gssi.it | §antonia.bertolino@isti.cnr.it

Abstract—Test suite reduction approaches aim at decreasing software regression testing costs by selecting a representative subset from large-size test suites. **Most existing techniques are too expensive for handling modern massive systems** and moreover depend on artifacts, such as code coverage metrics or specification models, that are not commonly available at large scale. **We present a family of novel very efficient approaches** for similarity-based test suite reduction that apply algorithms borrowed from the big data domain together with smart heuristics for finding an evenly spread subset of test cases. The approaches are very general since they only use as input the test cases themselves (test source code or command line input). **We evaluate four approaches** in a version that selects a fixed budget B of test cases, and also in an adequate version that does the reduction guaranteeing some fixed coverage. The results show that the **approaches yield a fault detection loss comparable to state-of-the-art techniques**, while providing **huge gains in terms of efficiency**. When applied to a suite of more than 500K real world test cases, the most efficient of the four approaches could select B test cases (for varying B values) in less than 10 seconds.

Index Terms—Clustering, Random projection, Similarity-based testing, Software testing, Test suite reduction.

T' the changes made to S are considered (*modification-aware regression testing*) or not [34].

The proposed techniques have been evaluated and compared against each other using metrics relative to their fault detection effectiveness (e.g., the Average Percentage of Fault Detection of the reordered test suite, or the loss in faults detected by the reduced test suite T'); for test reduction and selection, also metrics relative to cost savings, e.g., the size or the execution time of T' are compared against those of the full suite T .

Another important factor that should be taken into account is the *cost of the technique itself*, both in terms of the computational effort and of the resources it requires. In other words, when evaluating whether investing on an automated approach aimed at reducing the cost of testing is worth, a complete cost-benefit analysis should also include the overheads implied by the approach [18].

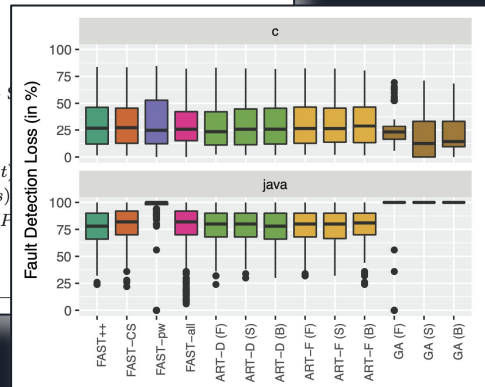
However, not many of the proposed techniques have considered such implied costs. In 2004, Orso and coauthors already noticed that in regression testing efficiency and precision need

Algorithm 1 FAST++

Input: Test Suite T ; Budget B

Output: Reduced Test Suite R

```
1:  $P \leftarrow \text{RandomProjection}(T)$ 
2:  $s \leftarrow \text{FirstSelection}(P)$ 
3:  $R \leftarrow \text{List}(s)$ 
4:  $D \leftarrow \text{Distance}()$ 
5:  $D(s) \leftarrow 0$ 
6: while ( $\text{Size}(R) < B$ ) do
7:   for all  $t \in P$  do
8:     if  $d(P(t), P(s))^2 < D(t)$ 
9:        $D(t) \leftarrow d(P(t), P(s))$ 
10:   $s \leftarrow \text{ProportionalSample}(P)$ 
11:   $R \leftarrow \text{Append}(R, s)$ 
12:   $D(s) \leftarrow 0$ 
13: return  $R$ 
```



Examples of popular empirical studies types: **Algorithm Evaluation**

Scalable Approaches for Test Suite Reduction

Emilio Cruciani*, Breno Miranda†§, Roberto Verdecchia*‡, and Antonia Bertolino§

*Gran Sasso Science Institute | L'Aquila, Italy

†Federal University of Pernambuco | Recife, Brazil

‡Vrije Universiteit Amsterdam | Amsterdam, The Netherlands

§ISTI – Consiglio Nazionale delle Ricerche | Pisa, Italy

*emilio.cruciani@gssi.it | †bafm@cin.ufpe.br | ‡roberto.verdecchia@gssi.it | §antonia.bertolino@isti.cnr.it

Abstract—Test suite reduction approaches aim at decreasing software regression testing costs by selecting a representative subset from large-size test suites. **Most existing techniques are too expensive for handling modern massive systems** and moreover depend on artifacts, such as code coverage metrics or specification models, that are not commonly available at large scale. **We present a family of novel very efficient approaches** for similarity-based test suite reduction that apply algorithms borrowed from the big data domain together with smart heuristics for finding an evenly spread subset of test cases. The approaches are very general since they only use as input the test cases themselves (test source code or command line input). **We evaluate four approaches** in a version that selects a fixed budget B of test cases, and also in an adequate version that does the reduction guaranteeing some fixed coverage. The results show that **the approaches yield a fault detection loss comparable to state-of-the-art techniques**, while providing **huge gains in terms of efficiency**. When applied to a suite of more than 500K real world test cases, the most efficient of the four approaches could select B test cases (for varying B values) in less than 10 seconds.

Index Terms—Clustering, Random projection, Similarity-based testing, Software testing, Test suite reduction.

T' the changes made to S are considered (*modification-aware* regression testing) or not [34].

The proposed techniques have been evaluated and compared against each other using metrics relative to their fault detection effectiveness (e.g., the Average Percentage of Fault Detection of the reordered test suite, or the loss in faults detected by the reduced test suite T'); for test reduction and selection, also metrics relative to cost savings, e.g., the size or the execution time of T' are compared against those of the full suite T .

Another important factor that should be taken into account is the *cost of the technique itself*, both in terms of the computational effort and of the resources it requires. In other words, when evaluating whether investing on an automated approach aimed at reducing the cost of testing is worth, a complete cost-benefit analysis should also include the overheads implied by the approach [18].

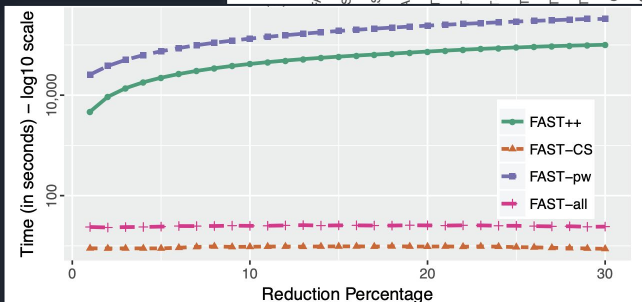
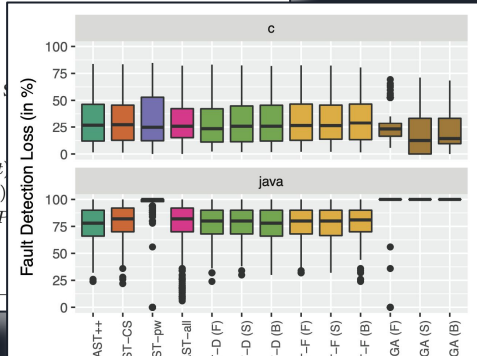
However, not many of the proposed techniques have considered such implied costs. In 2004, Orso and coauthors already noticed that in regression testing efficiency and precision need

Algorithm 1 FAST++

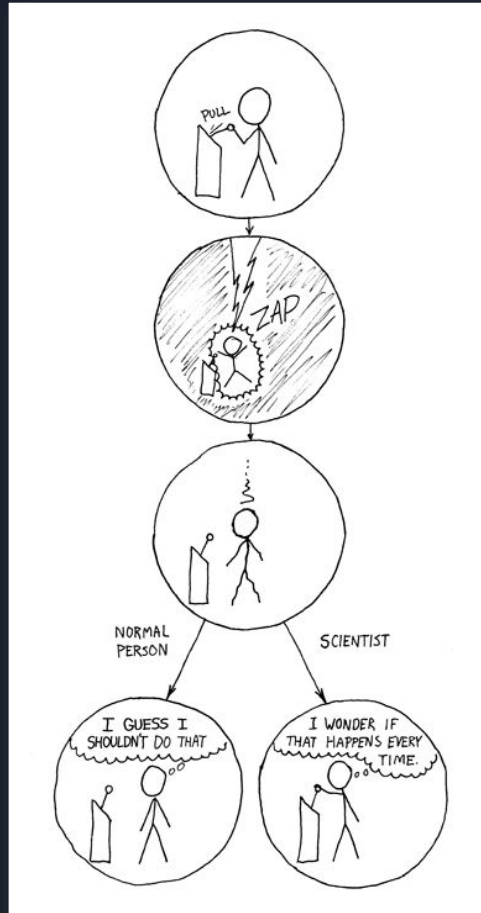
Input: Test Suite T ; Budget B

Output: Reduced Test Suite R

```
1:  $P \leftarrow \text{RandomProjection}(T)$ 
2:  $s \leftarrow \text{FirstSelection}(P)$ 
3:  $R \leftarrow \text{List}(s)$ 
4:  $D \leftarrow \text{Distance}()$ 
5:  $D(s) \leftarrow 0$ 
6: while ( $\text{Size}(R) < B$ ) do
7:   for all  $t \in P$  do
8:     if  $d(P(t), P(s))^2 < D(t)$ 
9:        $D(t) \leftarrow d(P(t), P(s))$ 
10:   $s \leftarrow \text{ProportionalSample}(P)$ 
11:   $R \leftarrow \text{Append}(R, s)$ 
12:   $D(s) \leftarrow 0$ 
13: return  $R$ 
```



A zoom in: Empirical Experiments



A zoom in: Empirical Experiments





“What if half of my servers crashed?”

What is the factor?
Which other factors could I
consider?

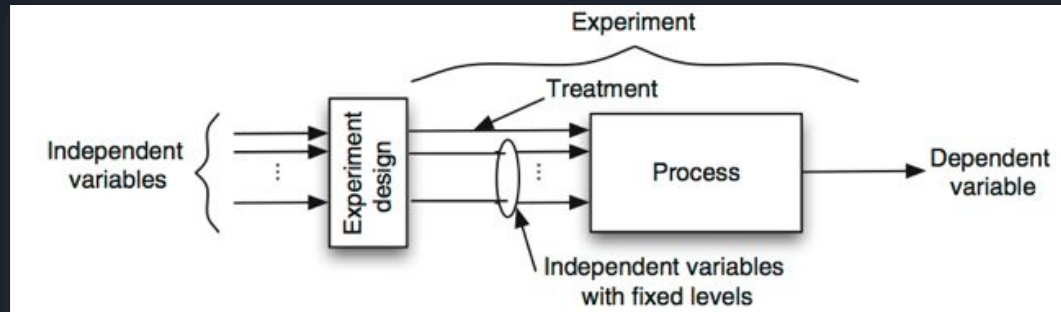
A zoom in: Empirical Experiments

Def: an empirical enquiry that **manipulates one factor or variable** of the **studied setting**

Identify and understand the **variables that play a role** in software development, and the **connections between variables**

Learn **cause-effect relationships** between the **development process** and the **obtained products**

Establish laws and theories about software construction that explain development behaviour

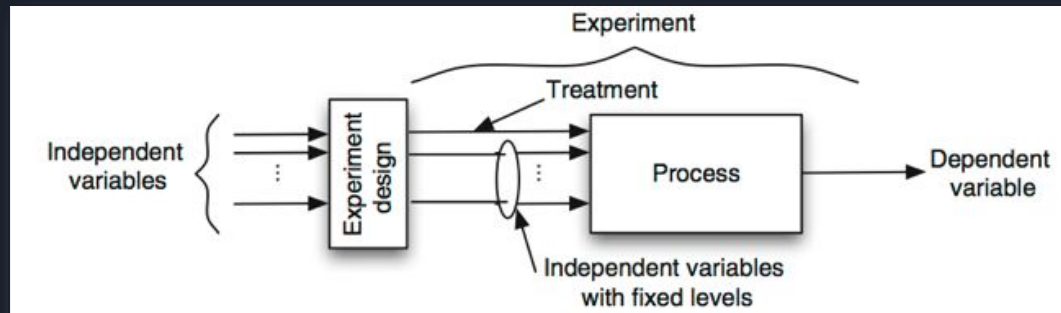


A zoom in: Empirical Experiments

A more intuitive definition of “experiment”:

Models **key characteristics** of a reality in a **controlled environment** and **manipulating them iteratively** to investigate the **impact of such variations** and get a better understanding of a phenomenon.

Laboratory: Simplified and controllable reality where the **phenomenon under study can be manipulated**

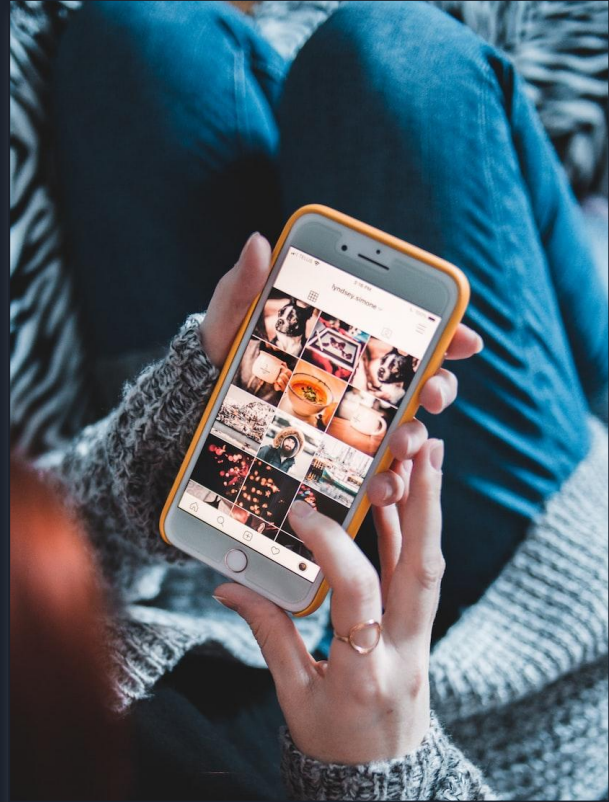


Experiments: Running example

Image encoding

How do different image encoding algorithms impact performance?

- PNG
- JPEG
- ...



Theory



If I use different image encoding algorithms...

... different performance



JPEG, PNG

measured performance per app

Observation

Experiments: Terminology

Independent variables: quantities that we are able to **manipulate/control** (a.k.a. input variables)

- e.g., *encoding algorithm, size of image, operating system, ...*

Dependent variables: the quantities **observed** in the study (a.k.a. response, output variables)

- e.g., *performance*

I want also to consider energy consumption, is it a dependent or independent variable?

I want also to consider
file size, is it a dependent
or independent variable?

Experiments: Terminology

Experimental subject: who applies the treatment

e.g. software developer, software architect, user

Experimental object: the receiver of the treatment

- *e.g., software application*

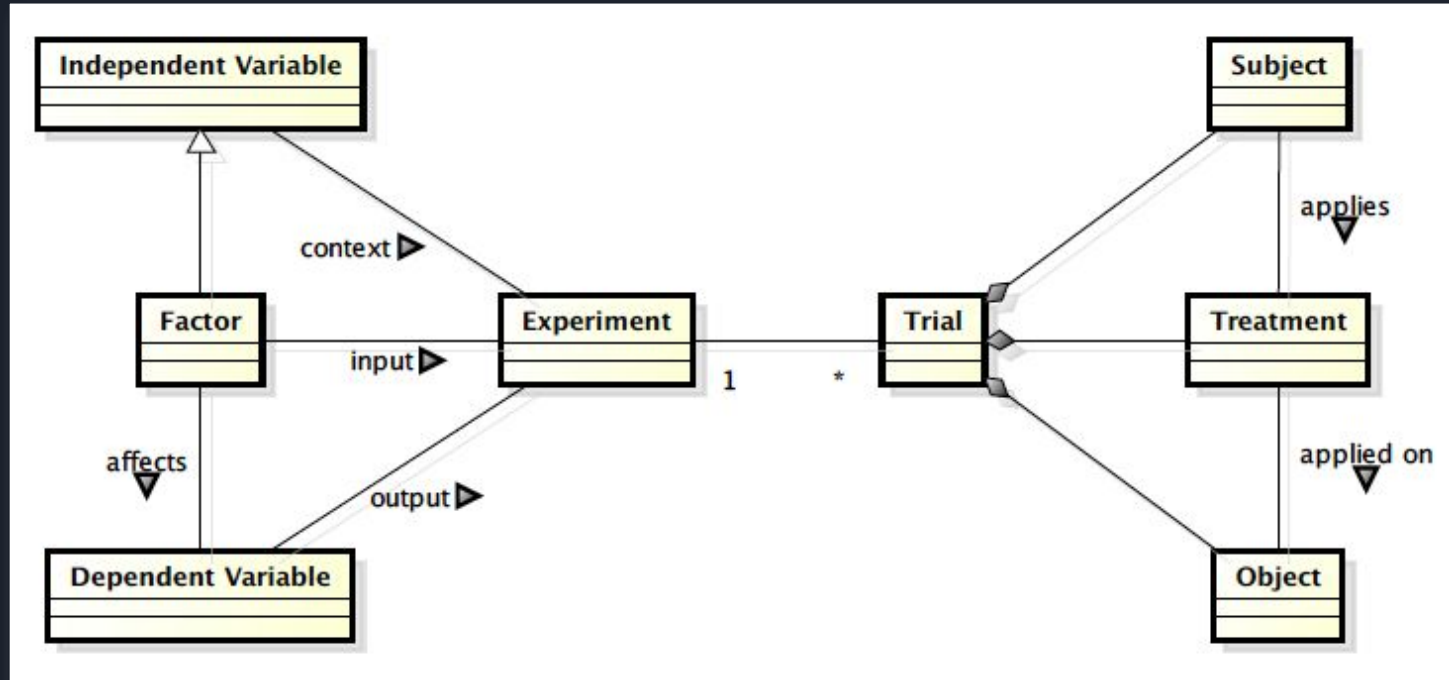
Experiments: Terminology

Trial: combination of **subject, object and treatment**

- *e.g. developer John develops app X that encodes images using the JPEG algorithm*

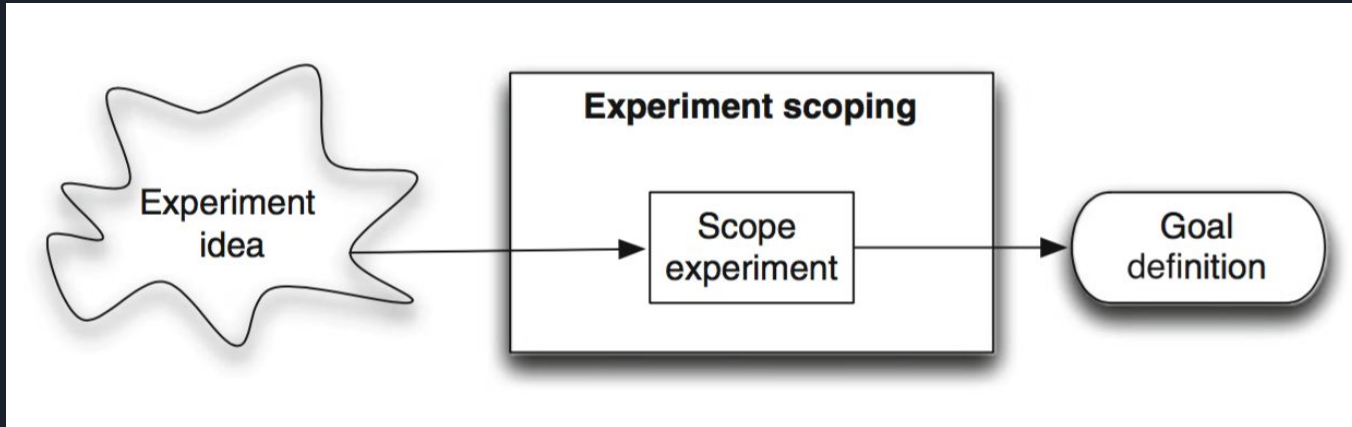
Experiment: a **combination of several trials** to observe the effects of the treatments

Experiments: Terminology, all together now!



How to start: Experimental scoping

- **Define the goal** of the study



Research approach: Goal-Question-Metric [1]

[1] V.R. Basili, "Software Modeling and Measurement: The Goal Question Metric Paradigm," Computer Science Technical Report Series, CS-TR-2956 (UMIACS-TR-92-96), University of Maryland, College Park, MD, September 1992.

How to start: Experimental scoping

*Conceptual
level*

Goal

*Operational
level*

Question

Question

Question

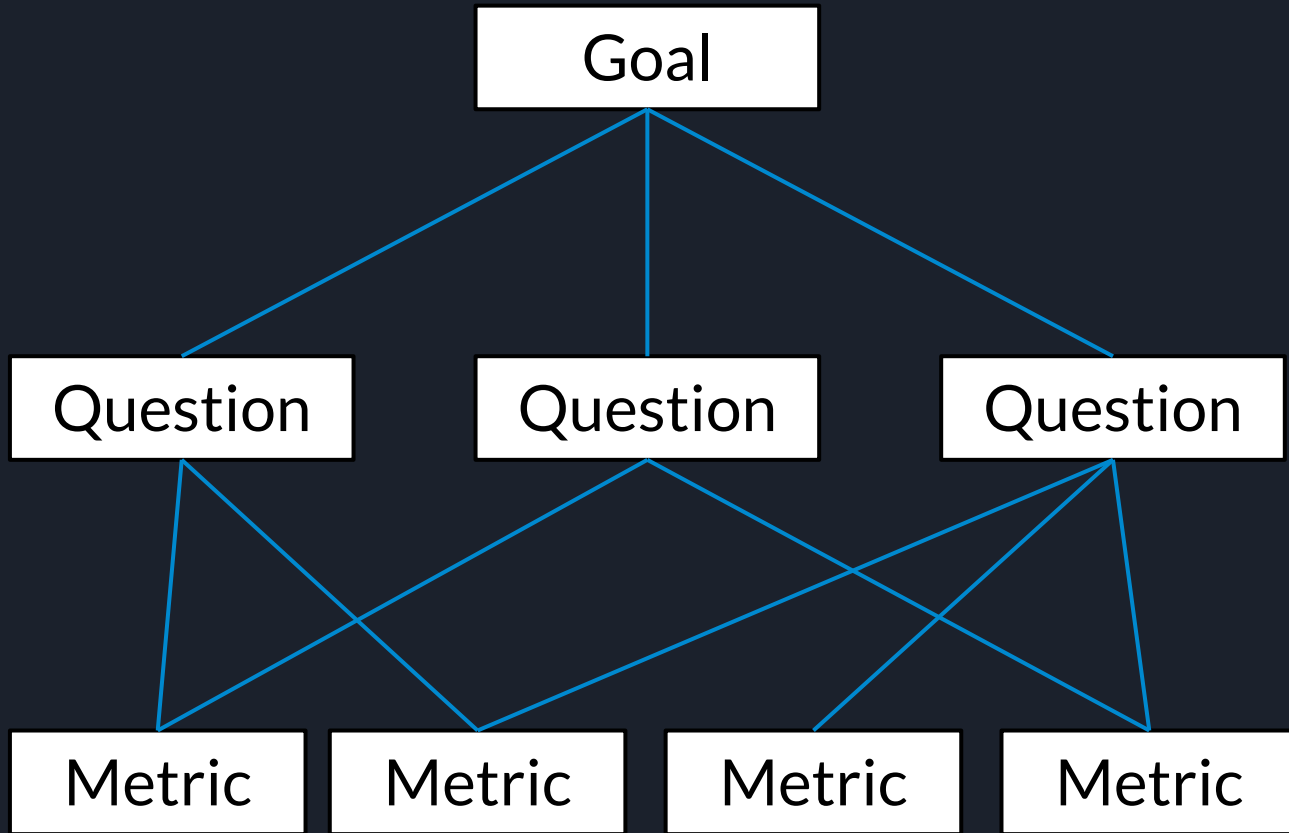
*Quantitative
level*

Metric

Metric

Metric

Metric



How to define the goal?

<i>Analyze</i>	Object(s) of study
<i>for the purpose of</i>	Purpose
<i>with respect to their</i>	Quality focus
<i>from the point of view of</i>	Perspective
<i>in the context of</i>	Context

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

Analyze

for the purpose of

with respect to their

from the point of view of

in the context of

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

<i>Analyze</i>	Image encoding algorithms
<i>for the purpose of</i>	
<i>with respect to their</i>	
<i>from the point of view of</i>	
<i>in the context of</i>	

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

<i>Analyze</i>	Image encoding algorithms
<i>for the purpose of</i>	Evaluation
<i>with respect to their</i>	
<i>from the point of view of</i>	
<i>in the context of</i>	

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

<i>Analyze</i>	Image encoding algorithms
<i>for the purpose of</i>	Evaluation
<i>with respect to their</i>	Performance
<i>from the point of view of</i>	
<i>in the context of</i>	

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

<i>Analyze</i>	Image encoding algorithms
<i>for the purpose of</i>	Evaluation
<i>with respect to their</i>	Performance
<i>from the point of view of</i>	Software Developers
<i>in the context of</i>	

How to define the goal?

Object of study	Purpose	Quality focus	Perspective	Context
Product	Characterize	Effectiveness	Developer	<depends on experiment>
Technology	Monitor	Energy consumption	Maintainer	
Technical choice	Evaluate	Cost	Project manager	
Process	Predict	Reliability	Corporate manager	
Model	Control	Maintainability	Customer	
Metric	Change	Portability	User	
Theory		Performance	Researcher	

How to define the goal? **Image encoding example**

<i>Analyze</i>	Image encoding algorithms
<i>for the purpose of</i>	Evaluation
<i>with respect to their</i>	Performance
<i>from the point of view of</i>	Software Developers
<i>in the context of</i>	Android mobile apps

Experiments: research question formulation

- **Divide** the goal of the study in **smaller “chunks”**, i.e., **research questions** that can be **independently assessed**
- The **experimental results** will be the **answers to our questions**
- **Example:**
 - **Goal:** *Evaluate the performance of image compression algorithms*
 - **RQ1: ???**

Experiments: research question formulation

- **Divide** the goal of the study in **smaller “chunks”**, i.e., **research questions** that can be **independently assessed**
- **Example:**
 - **Goal:** *Evaluate the performance of image compression algorithms*
 - RQ1: Do **different compression algorithms** affect performance?
 - RQ2: **what if we consider file size?**

Experiments: research question formulation

- **Divide** the goal of the study in **smaller “chunks”**, i.e., **research questions** that can be **independently assessed**
- **Example:**
 - **Goal:** *Evaluate the performance of image compression algorithms*
 - RQ1: Do **different compression algorithms** affect performance?
 - RQ2: Do **different image sizes** affect performance?

Experiments: research question formulation

- Each research question has a **null** and an **alternative hypothesis**
- Example:
 - RQ1: Do **different compression algorithms** affect performance?
 - **Null hypothesis (H_0): No statistically significant difference** in the algorithms' performance can be noted
 - **Alternative hypothesis (H_a): A statistically significant difference** in the algorithms' performance can be noted

Experimental object and treatment selection

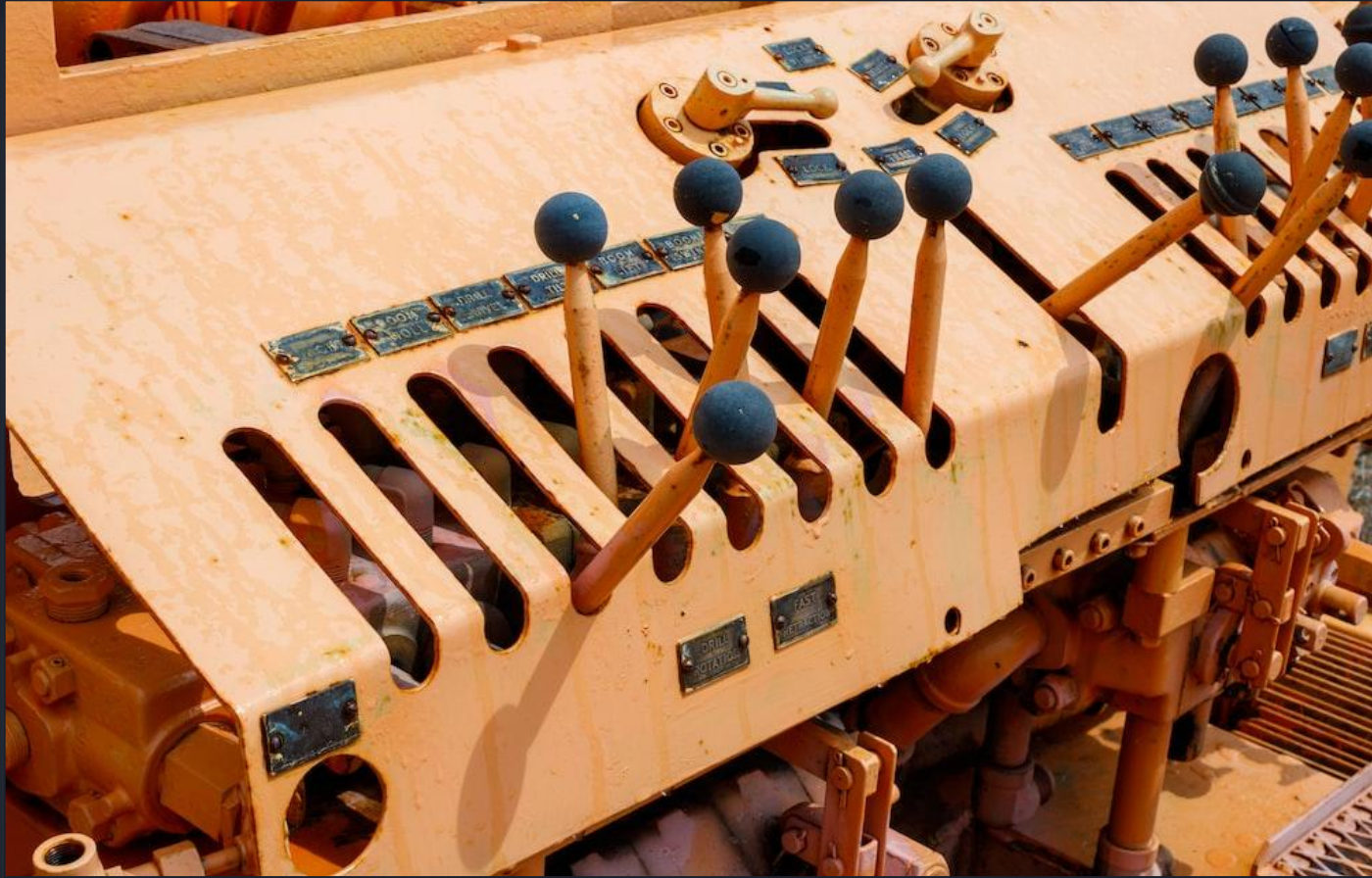
- Identify **significant objects** and **treatments**
- Example:
 - **Object: Standard images (e.g., “Lenna”)**
 - **Treatments**
 - **State of the art algorithms**
 - **Realistic file sizes**
- **NOTE:** objects and treatments should always be carefully selected
...are they really representative of the real world?



Combination of object and treatment into experimental runs

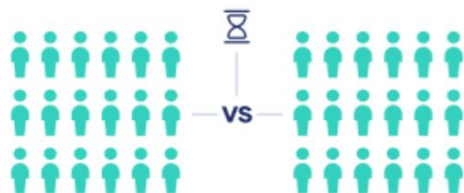
- **Combine** the **objects and treatments** according to a strategy
- Each combination is an **experimental run**
- In the best case scenario, use **all combinations**
- If not possible, select **statistically representative combinations** (e.g., via latin square)
- Example of exhaustive combination
 - **Each compression algorithm** is run on **all images** and **all sizes**

Experiment execution!



Data analysis

Paired-samples t test



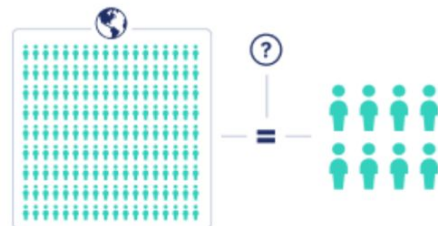
Investigate whether there's a difference within a group between two points in time (within-subjects).

Independent-samples t test



Investigate whether there's a difference between two groups (between-subjects).

One-sample t test



Investigate whether there's a difference between a group and a standard value or whether a subgroup belongs to a population.

Data analysis

- After experiment execution
- **Statistically analyze** the **data collected** (e.g., image compression time) to **derive results**
- Some common examples of data analysis techniques
 - **Normality assessment** (precondition)
 - **T-test** (compare means of two groups)
 - **Spearman's rank** (correlation between dependent and independent variable)
 -

Data analysis

Data-Centric Green AI An Exploratory Empirical Study

Verdecchia*, Luís Cruz[†], June Sallou[‡], Michelle Lin[§], James Wickenden[¶], Estelle Hotellier^{||}
*Vrije Universiteit Amsterdam, The Netherlands - r.verdecchia@vu.nl
[†]Delft University of Technology, The Netherlands - l.cruz@tudelft.nl
[‡]IRISA - UMR 6074 F-35000 Rennes, France - june.benvegna-sallou@irisa.fr
[§]McGill University, Canada - michelle.lin2@mail.mcgill.ca
[¶]University of Bristol, United Kingdom - jw17943@bristol.ac.uk
^{||}IRISA, France - estelle.hotellier@inria.fr

Datasets, been estimated to consume the energy equivalent of a transatlantic American flight [5]. Hence, a new subfield is emerging to make the development and application of AI technology environmentally sustainable: *Green AI* [6].

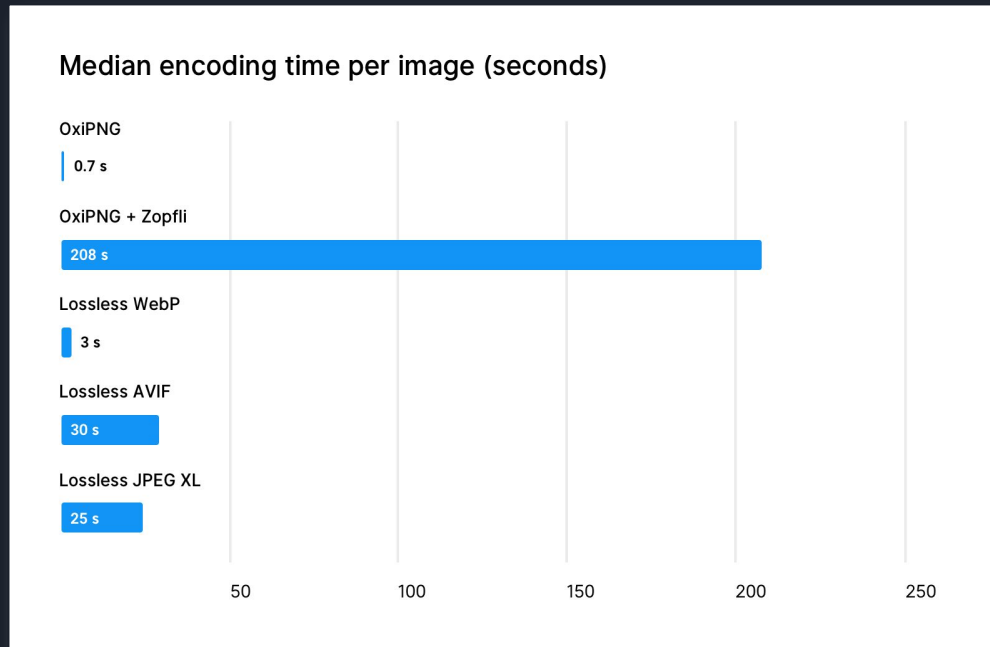
In a related note, the current research practice of collecting large amounts of data is not necessarily yielding better results. Being able to collect high-quality data is more important than collecting big data – a trend coined as *Data-centric AI*. This data-centric AI focuses on leveraging existing learning techniques that squeeze even more out of efficient practices to collect high-quality data.

We conduct an exploratory empirical study on Green AI and Data-centric AI. We explore the impact of modifying dataset characteristics on training AI models. We investigate the impact of AI the branch of AI that focuses on sampling and analyzing data on sample size, rangeability, and energy consumption.

As a preliminary step, in order to assess if the energy consumption (DV_1) data we collected is normally distributed, we carry out a visual normality assessment by means of quantile-quantile (Q-Q) plot, followed by a Shapiro-Wilk normality test. From the inspection of the generated Q-Q plot, and the Shapiro-Wilk test result ($W=0.52$ and $p\text{-value}=2.2e-16$), we can confidently conclude that the data collected is not normally distributed. Hence, for each sub-experiment, we sample the data gathered in the run reporting the median energy consumption value. Subsequently, in order to evaluate if a correlation exists between our dependent and independent variables, we leverage the calculation of the one-tailed Spearman's rank correlation coefficient (ρ). We adopt Spearman's ρ as it provides a non-parametric measure, and can be used to calculate the potential correlation between our ordinal (IV_1 - IV_3) and continuous variables (DV_1 and DV_2). Finally, to provide further insights into our results, we calculate percentage changes to summarize the difference in energy consumption and F1-scores between different algorithms, number of data points, and number of features.

Draw conclusions and apply to practice

*“From the findings, I can scientifically conclude that **OxiPNG is the best algorithm** I can use if **performance** is my primary goal”*



Empirical Software Engineering:

The world is our playground

- How does **coffee** affect the productivity of programmers?
- What programming language is more prone to have **bugs**?
- What programming language consumes **more energy**?
- Are microservice architectures **harder to maintain**?
- ...



We still have time!

Let's make the best out of it!

- **Option 1**

Empirical experiments are hard!

Let's look at one together!

https://robertoverdecchia.github.io/papers/ICT4S_2022_1.pdf

- **Option 2**

Got the empirical experiment basics?

Let's design a study together!